

Plan 4 — Technical Integrations + Build Sequence

Status: Locked 2026-04-24 **Companion docs:** [plan-1-auth.md](#), [plan-2-retention.md](#), [plan-3-integration-scope.md](#), [plan-5-llm-cost.md](#)

Executive summary

Each external system is wired through a small `IntegrationAdapter` interface (one TypeScript file per provider). OAuth grants are stored encrypted via AES-256-GCM in `integration_grants`. A single in-process scheduler runs per-tenant fetchers on a cadence; webhook providers POST to a single endpoint per provider, with HMAC-based tenant identification and dedup. Every fetcher reads `INTEGRATION_SCOPE` from Plan 3 to decide which fields to pull, normalizes results to a flat `integration_signals` table, and the existing competency scorer reads from there. Adding a new integration is one file + one entry in the adapter registry. Pilot ships with xlsx upload only; Phase 2 wave 1 (BambooHR + Salesforce + Calendar, 10 eng days) follows the law-firm pilot. Wave 2 (GitHub + Slack + Jira + Rippling, 17 days) targets engineering-heavy customers.

Locked decisions

#	Decision
1	OAuth library: arctic (lightweight, framework-agnostic)
2	Token encryption: env-var master key for pilot ; AWS KMS at first enterprise customer who asks
3	Webhook secret per tenant: rotated only on demand (no scheduled rotation)
4	Rate-limit strategy when exhausted: skip and retry next cycle
5	GitHub: start as OAuth App for first customer , migrate to GitHub App when 5+ customers
6	Slack: build as custom workspace app first , submit for public review when 3+ customers using it
7	Reconciliation pull frequency: daily at 02:00 UTC
8	Phase 2 wave 1 sequence: BambooHR first (alone), then Salesforce + Calendar in parallel
9	Integration request workflow: collect requests, no public roadmap
10	Sandbox accounts at every connected provider for contract tests

The IntegrationAdapter interface

```

```ts // lib/integrations/types.ts export interface IntegrationAdapter { id: string; // 'bamboohr', 'salesforce', etc.
display_name: string; category: "HRIS" | "CRM" | "VCS" | "ITS" | "Comms" | "Calendar" | "LMS" | "Perf" | "Time" |
"Docs";

// OAuth lifecycle buildAuthorizationURL(opts: { tenantid: string; state: string; redirecturi: string }): URL;
exchangeCodeForTokens(opts: { code: string; redirect_uri: string }): Promise<TokenSet>;
refreshAccessToken(grant: IntegrationGrant): Promise<TokenSet>;

// Data fetching (called by the scheduler) pull(grant: IntegrationGrant, since: Date):
Promise<NormalizedSignal[]>;

// Webhooks (real-time providers only) verifyWebhookSignature?(req: Request, secret: string): boolean;
handleWebhook?(payload: unknown, grant: IntegrationGrant): Promise<NormalizedSignal[]>;

// Health testConnection(grant: IntegrationGrant): Promise<{ ok: boolean; reason?: string }>;

// Rate-limit declaration (used by runner) ratelimit: { requestsperhour: number; scope: "pertoken" | "per_tenant" |
"global" }; }

export type NormalizedSignal = { employeekey: string; signaltype: string; value: number | string; unit?: string;
recordedat: string; // ISO 8601 UTC sourceeventid?: string; // for idempotency rawmeta?: Record<string,
unknown>; // limited per Plan 3 allowlist }; ```

```

Adding GitHub: implement that interface, register in central INTEGRATIONS map, schema migration adds nothing (the `integration_signals` table is generic). One PR per integration.

## Schema additions

```

```sql CREATE TABLE integrationgrants ( id TEXT PRIMARY KEY, tenantid TEXT NOT NULL REFERENCES
tenants(id), integrationid TEXT NOT NULL, grantedbyuserid TEXT NOT NULL REFERENCES users(id),
grantedat TEXT NOT NULL, status TEXT NOT NULL DEFAULT 'active', -- 'active' | 'revoked' | 'error' |
'pendingreconsent' scopes TEXT NOT NULL, -- JSON: actual OAuth scopes granted optinfields TEXT NOT
NULL, -- JSON: which OPTIN/PULL fields enabled encryptedaccesstoken BLOB NOT NULL,
encryptedrefreshtoken BLOB, tokenexpiresat TEXT, webhooksecret BLOB, -- per-tenant per-provider HMAC
secret lastpullat TEXT, lastpullstatus TEXT, -- 'ok' | 'error' lastpullerror TEXT, UNIQUE(tenantid, integration_id)
);

```

```

CREATE TABLE integrationsignals ( id INTEGER PRIMARY KEY AUTOINCREMENT, tenantid TEXT NOT
NULL, employeekey TEXT NOT NULL, integrationid TEXT NOT NULL, signaltype TEXT NOT NULL,
valuenumeric REAL, valuetext TEXT, unit TEXT, recordedat TEXT NOT NULL, -- when the event happened in
source ingestedat TEXT NOT NULL, sourceeventid TEXT, -- idempotency key from source expiresat TEXT
NOT NULL, -- per Plan 2 retention policy UNIQUE(tenantid, integrationid, sourceeventid) ); CREATE INDEX
idxsignalstenantemployee ON integrationsignals(tenantid, employeekey); CREATE INDEX idxsignalsexpires
ON integrationsignals(expiresat);

```

```

CREATE TABLE webhookevents ( id TEXT PRIMARY KEY, integrationid TEXT NOT NULL, tenantid TEXT, --
resolved from HMAC signaturevalid INTEGER NOT NULL, payloadhash TEXT NOT NULL, -- for replay
detection receivedat TEXT NOT NULL, processedat TEXT, status TEXT, -- 'pending' | 'ok' | 'error' | 'duplicate'
error TEXT ); CREATE UNIQUE INDEX idxwebhookpayloadhash ON webhookevents(integrationid,
payload_hash); ```

```

OAuth + token security

Library: arctic for OAuth flow (URL building, code exchange). Each provider's official SDK for API calls (@octokit/rest, jsforce, @slack/web-api, etc.).

Token encryption:

```
``ts // lib/integrations/crypto.ts const masterKey =  
Buffer.from(env.SKILLNEX_TOKEN_KEY_HEX, "hex"); // 32 bytes export function  
encryptToken(plaintext: string): Buffer { /* iv + ciphertext + auth tag */ }  
export function decryptToken(blob: Buffer): string { /* reverse */ } ``
```

Master key in DigitalOcean App Platform env vars (DO encrypts at rest). Key rotation supported: previous_key env var attempted as fallback if current_key decrypt fails, automated re-encrypt on first access. Migration to AWS KMS documented for Year-1 enterprise customers per privacy guide §Customer-Managed Keys.

OAuth state CSRF protection: state token = HMAC of tenant_id + nonce + ttl, signed with server secret. Validated on callback. No state stored in DB (stateless flow). 10-minute TTL.

Webhook architecture

Single endpoint per provider: /api/webhooks/{provider} (no tenant in URL). Tenant identified by HMAC signature against per-tenant webhook_secret from integration_grants.

Flow:

- Provider POSTs payload + signature header
- Endpoint extracts integration_id from URL
- Loops through all active grants for that integration_id, tries each webhook_secret against the HMAC header (constant-time compare)
- Match → validates payload, deduplicates via webhook_events.payload_hash, calls adapter.handleWebhook(payload, grant)
- No match → 401, log to webhook_events with signature_valid=0 for monitoring (could be probe traffic)

For high-volume providers at ~100+ tenants per provider, the brute-force HMAC loop becomes a problem. At that point, add a webhook_routing index by signature prefix. Not a pilot concern.

Deduplication: every webhook payload hashed, stored in webhook_events. Same hash within 24 hours = duplicate, return 200 without re-processing.

Background runner

Pilot architecture: in-process Node scheduler on the single DO Droplet.

```
``ts // lib/integrations/runner.ts import cron from "node-cron";  
  
cron.schedule("/15 *", async () => { // Every 15 min: pull from polling-only integrations (Calendar, BambooHR, LMS) await runPollingIntegrations(); });  
  
cron.schedule("0 2 *", async () => { // Daily 02:00 UTC: full reconciliation pull await runFullReconciliation(); });
```

```
cron.schedule("0 /6 ", async () => { // Every 6 hours: retention sweep (Plan 2) await runRetentionSweep(); }); ``
```

Failure handling: per-tenant per-integration. One tenant's failed pull doesn't block others. Failures land in `integration_grants.last_pull_error` + audit log. Three consecutive failures → status = `error`, email Owner.

Migration path (when ~50 tenants or reliability issues): replace `node-cron` with BullMQ + Redis (DO Managed Redis ~\$15/mo). Same per-tenant fetcher functions, just enqueued instead of called inline. The `IntegrationAdapter` interface doesn't change.

Rate limits

Each provider's adapter declares its rate limits in code:

```
``ts export const githubAdapter: IntegrationAdapter = { rate_limit: {  
  requests_per_hour: 5000, scope: "per_token" }, // ... }; ``
```

Token bucket per `(tenant_id, integration_id)`. When exhausted, runner skips that tenant's pull for the cycle, retries next interval. Logged.

Per-integration cost matrix

Integration	OAuth complexity	Webhooks	Eng days	Phase	Reason
BambooHR	Simple OAuth, public docs	No, daily batch	3	2.1	Every customer has HRIS. Replaces xlsx pain. Free tier API.
Workday	Enterprise SAML + OAuth	No, EIB scheduled	8-10	3	Bespoke contracts; high friction. Defer until enterprise demands.
Rippling	OAuth + partner program	Yes	4	2.2	Cross-department in one API.
Salesforce	Standard OAuth	Yes (Platform Events)	5	2.1	Sales-heavy customers. Maps to multiple competencies.
GitHub	OAuth App → migrate to GitHub App	Yes	3	2.2	Engineering customers. Simple API.
GitLab	Same shape as GitHub	Yes	2	3	After GitHub, mostly same code.

Integration	OAuth complexity	Webhooks	Eng days	Phase	Reason
Jira	OAuth via Atlassian	Yes	5	2.2	Engineering + ops. Webhooks quirky.
Linear	Modern API	Yes	2	3	Smaller market than Jira.
Slack	Custom app first → public review later	Yes	5	2.2	Plan 3 limits us to metadata, simplifying build. App review is calendar-time, not eng-time.
MS Teams	Microsoft Graph	Yes	6	3	Bigger eng load; smaller mid-market share.
Google Calendar	Google OAuth	No (FreeBusy poll)	2	2.1	Cross-team meeting metric is high-value, low-risk.
M365 Calendar	Microsoft Graph	No	3	3	Often required if Outlook is the calendar.
Lattice	Partner API	No	3	3	Not all customers have it. Comparison-column source.
15Five	Public API	No	2	3	Same as Lattice.
Workday Learning / Coursera Business / Udemy	Each different	No	3 per	4	Build per customer ask.
Toggl / Harvest / Clockify	Simple OAuth	No	2 per	4	Per customer ask.
Notion / Confluence	Simple OAuth	No	2 per	4	Per customer ask.

Wave 1 total: BambooHR + Salesforce + Calendar = 10 eng days. Wave 2 total: GitHub + Slack + Jira + Rippling = 17 eng days.

Recommended sequence

- **We receive, processing fails** (DB error, transform bug) → payload stored in `webhook_events` regardless. Retry job replays failed events for 24 hours.
- **Provider replays same event** (network retry) → `payload_hash` UNIQUE constraint catches duplicate, returns 200, no-op.

Standard webhook hygiene. Implements once, every integration benefits.

State Bar / law firm pilot — exclusion enforcement

Per privacy guide §Law Firm Pilot: *"Do not touch partner-level data in the pilot."* Every integration adapter respects an `excluded_employee_keys` filter passed by the runner. Skillnexus's `employees.excluded_from_review` flag (Plan 1) is honored at fetch time — those employees are skipped, no records ever land in `integration_signals`.

For the pilot specifically: integrations are **not turned on**. Law firm uploads xlsx. Once pilot succeeds, Phase 2 opens with their HRIS as the first integration.

Build sequence (15 days for Wave 1, +17 for Wave 2)

Day	Work
1	Schema migration: <code>integration_grants</code> , <code>integration_signals</code> , <code>webhook_events</code> . Encryption helpers. OAuth state HMAC.
2	IntegrationAdapter interface + central INTEGRATIONS map + scheduler skeleton. Generic <code>/api/oauth/{provider}/start</code> and <code>/callback</code> .
3	<code>/api/webhooks/{provider}</code> generic endpoint with HMAC + dedup + retry.
4	<code>/app/integrations</code> connected/disconnected UI + per-integration status page.
5-7	BambooHR adapter (3 days) — consent screen rendering from <code>INTEGRATION_SCOPE</code> , <code>pull()</code> , daily batch in scheduler, test fixtures.
8-12	Salesforce adapter (5 days) — Platform Events webhook subscription, real-time pulls.
13-14	Google Calendar adapter (2 days) — polling only, cross-team meeting computation.
15	End-to-end smoke: signup → connect BambooHR → connect Salesforce → connect Calendar → competency scores compute → narrative cites verified signals. Wave 1 done.
16+	Wave 2: GitHub, Slack, Jira, Rippling. ~17 days.

Tests that prove it works

Test	Verifies
tests/integrations/oauth-state.test.ts — replayed state token rejected	CSRF
tests/integrations/webhook-dedup.test.ts — same payload twice = one processing	Idempotency
tests/integrations/scope.test.ts — pull() output for each adapter contains only WILL_PULL fields	Plan 3 enforcement
tests/integrations/scope.test.ts — OPT_IN_PULL fields appear only when grant has them	Three-tier model
tests/integrations/runner.test.ts — failed tenant doesn't block others	Failure isolation
tests/integrations/runner.test.ts — disconnect → grant soft-deleted + signals scheduled for purge	Right-to-deletion path
tests/integrations/excluded.test.ts — employee with excluded_from_review=1 produces zero signals	Partner-data exclusion
Contract test (daily, against provider sandboxes) — adapter still parses provider response	API drift detection
tests/integrations/rate-limit.test.ts — exhausted bucket → skip, retry next cycle	Rate-limit handling

Items deferred

- BullMQ + Redis migration — when reliability or scale demands
- GitHub App migration from OAuth App — at 5+ customers
- Slack public Marketplace listing — at 3+ customers
- Workday Partner program — at first enterprise customer demand
- Per-integration KMS keys — Year 1
- Salesforce AppExchange listing (~\$5K) — Year 1
- Per-tenant scope override config — post-pilot (Plan 3 decision #10)